
EMBEDDED HACKING

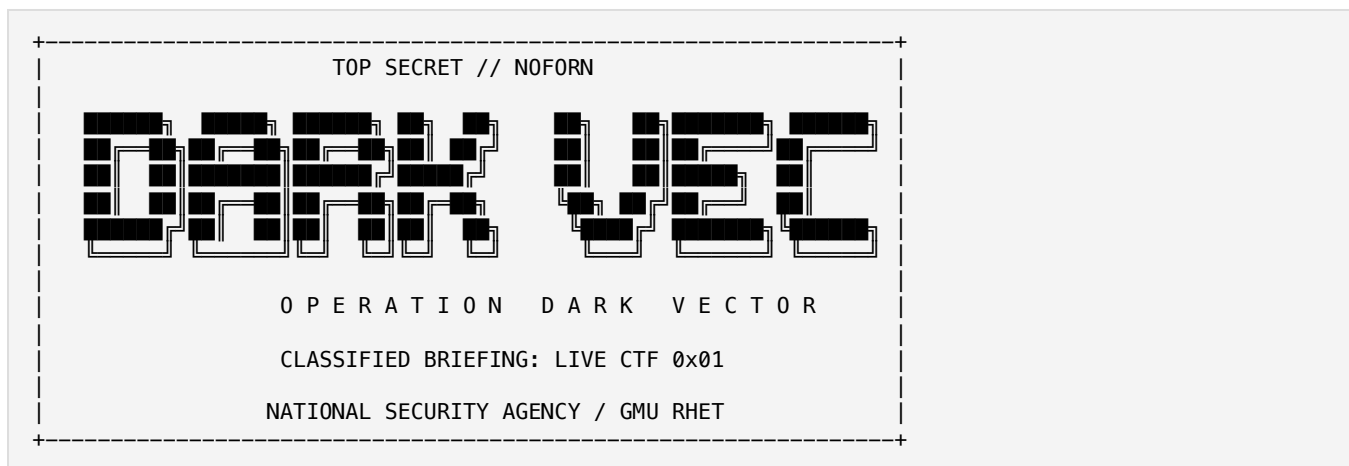
Reverse Engineering the Raspberry Pi Pico 2

Operation Dark Vector - Classified Intelligence Briefing

ARM Cortex-M33 • GDB • Ghidra

September 25, 2026

For educational and defensive security research only



1. Executive Intelligence Summary

At **04:17 EDT**, automated radio-frequency monitoring sensors operated by the National Security Agency (NSA) near **Centreville, Virginia** (vicinity of Route 28 and the Bull Run corridor: 38.840280° N, 77.428890° W) interdicted an unauthorized low-altitude micro-unmanned aerial vehicle (UAV). The craft suffered battery exhaustion due to severe head-winds and landed in wooded terrain.

Mounted beneath the carbon airframe was an **unidentified, hermetically sealed 6061-T6 aluminum canister** locked by an electromagnetic release latch. During its low-altitude ingress across Northern Virginia airspace, the craft operated under strict **RF Emission Control (EMCON / Radio Silence)**. To evade regional SIGINT radar grids, the drone utilized purely passive GPS satellite navigation with dead-reckoning flight guidance—no telemetry transmitter, no radio-control receiver, and no cellular uplink were active while airborne. A single-blade mock propeller attached to an SG90 servo oscillated continuously during flight to maintain aerodynamic stabilization.

The avionics core and flash memory were recovered intact and transported 12 miles east down Braddock Road to the **Embedded System Reverse Engineers at George Mason University (GMU)** Rapid Hardware Exploitation Team (Fairfax, VA).

The Discovery: Adversary Crash Recovery Beacon

Forensic hardware examination of the RP2350 avionics PCB revealed an onboard **REYAX RYLR998 915 MHz LoRa transceiver** wired to `UART1` (`GP8 TX`, `GP9 RX`). Avionics firmware analysis revealed the adversary's contingency protocol: while ingress flight was strictly radio-silent, the bootloader incorporates an **Emergency Crash Recovery Beacon**. If the drone suffers battery depletion or crashes, the firmware is programmed to awaken the LoRa module upon any subsequent power-on (`init_lora()`), broadcasting unencrypted coordinates over 915 MHz RF so adversary retrieval teams can track down the airframe.

When GMU engineers energized the RP2350 on the laboratory test bench, this recovery beacon immediately activated, broadcasting coordinates from the Centreville recovery origin (38.840280° N, 77.428890° W)!

The Primary Mission: SIGINT Intercept & Flight Diversion

Rather than suppressing the radio, GMU engineers **intercepted the RF downlink**:

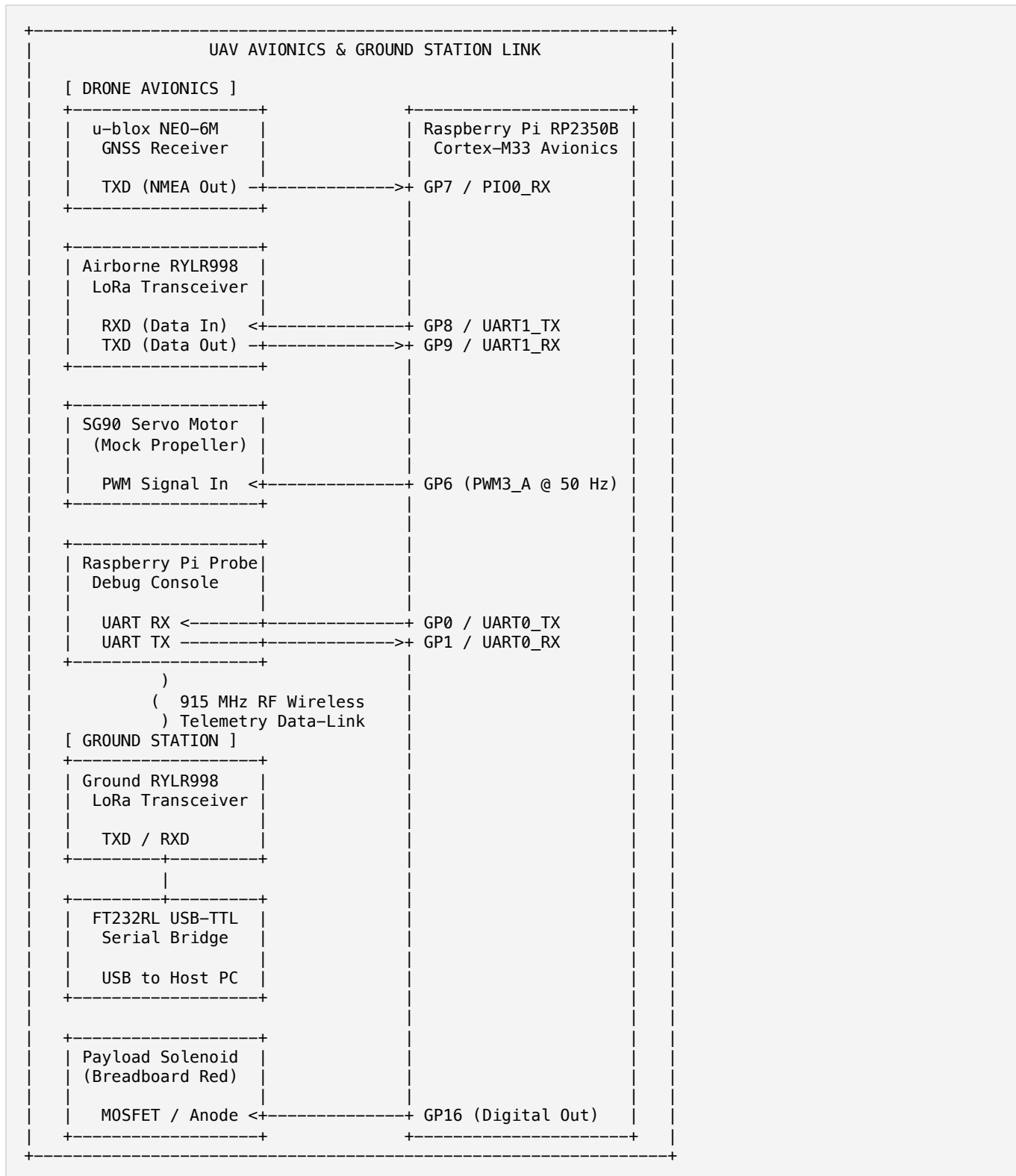
1. **Signal Intercept via FT232RL & LoRa:** GMU investigators provisioned a second REYAX RYLR998 on an FT232RL USB adapter, launching `scripts/telemetry_monitor.py` to eavesdrop on the live telemetry stream.
2. **Reverse Engineer `0x0011a_cb.bin` in Ghidra:** Identify the vector table, trace execution into `main()`, locate the guidance loop, and extract the hardcoded target coordinates in flash memory.
3. **Decode via Week 5 `scripts/float_hex_converter.py`:** Convert the literal pool quadwords into decimal degree coordinates using the Week 5 IEEE 754 utility.
4. **Identify the Intended High-Value Target:** Analyze decoded coordinates to reveal the intended target: the **National Reconnaissance Office (NRO) Headquarters** in Chantilly, Virginia (38.881940° N, 77.450280° W).
5. **Analyze RP2350 PIO GPS Architecture:** Examine how the avionics core uses a dedicated Programmable I/O (PIO) state machine to sample NMEA telemetry from the u-blox NEO-6M receiver on GP7 concurrently with dual hardware UARTs.
6. **Identify Propeller & Payload Actuators:** Locate the PWM propeller driver on GP6 and the payload release solenoid latch on GP16.
7. **Execute the Manual Binary Patch in Ghidra:** Overwrite the target coordinates directly in Ghidra with a safe offshore disposal waypoint in the **Atlantic Ocean** (37.000000° N, 74.000000° W), export the patched image, convert to UF2, and verify via the ground station HUD that the drone navigates far out to sea for safe payload neutralization.

2. Hardware Profile & Physical Teardown

Forensic hardware examination conducted at the GMU laboratory revealed a custom high-precision micro-avionics platform:

Forensic Parameter	Engineering Specification	Physical Dimensions / Mass
Airframe	3K Carbon Fiber unibody with PETG canopy	Wheelbase: 110 mm diagonal; 42 g
Flight Controller	Custom AIO board hosting Raspberry Pi RP2350B	25.5 mm x 25.5 mm; 4.8 g
Microcontroller	Dual ARM Cortex-M33 cores clocked at 150 MHz	QFN-80 package; 520 KB SRAM
Flash Memory	Winbond W25Q128JVPIM 16 MB QSPI NOR Flash	SOIC-8; Base: <code>0x10000000</code>
GNSS Module	u-blox NEO-6M GPS Receiver (Amazon)	Ceramic antenna; GP7 (PIO0_RX); 18 g
GNSS Interface	GPIO7 (PIO RX), 9600 baud, 8N1	Hardware-timed PIO state machine
Airborne LoRa RF	REYAX RYLR998 868/915MHz LoRa (Amazon)	UART1 (GP8 TX, GP9 RX), 115200 baud
Ground Station LoRa	2nd REYAX RYLR998 LoRa Module (Amazon)	Wireless telemetry receiver node
Ground USB Bridge	FT232RL USB to TTL Serial Adapter (Amazon)	Host PC USB interface (3.3V jumper)
Debug Console	UART0 (GP0 TX, GP1 RX), 115200 baud, 8N1	Raspberry Pi Debug Probe Serial Console

Forensic Parameter	Engineering Specification	Physical Dimensions / Mass
Mock Propeller	SG90 9g Micro Servo Motor (50 Hz PWM)	GP6 (PWM3 Channel A); 9 g; High-Speed
Payload Canister	Sealed 6061-T6 aluminum cylinder (mystery contents)	18 mm diameter x 36 mm; 45 g
Release Actuator	Micro-solenoid latch driven via MOSFET on GP16	Active HIGH pulse (Breadboard Red LED)
Power Supply	2S 450 mAh 75C LiPo battery pack (depleted)	60 mm x 18 mm x 15 mm; 29 g



3. Modular Repository Architecture

To conform with the standard course CTF software architecture (established in `0x0001b_ctf` and `0x0017a_ctf`), the avionics firmware is organized into a strictly modular layout separating header interfaces (`include/`), C source implementations (`src/`), and Python host tools (`scripts/`):

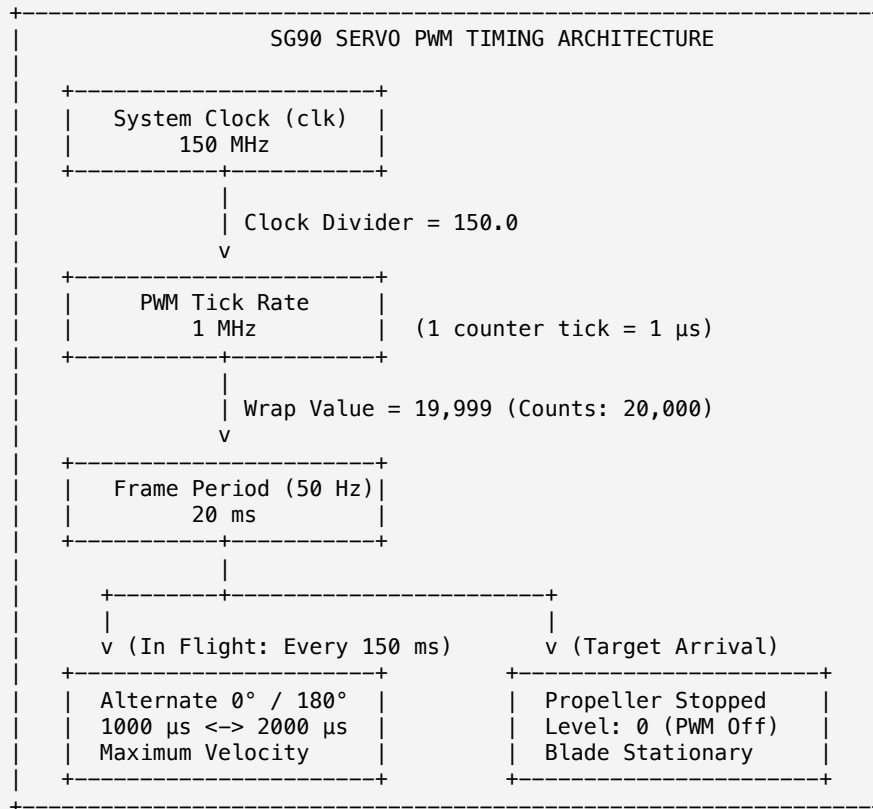
```

0x0011a_cb/
├── CMakeLists.txt           # Pico SDK RP2350 build configuration
├── pico_sdk_import.cmake    # SDK linkage definition
├── uart_rx.pio              # PIO0 state machine assembly for GPS RX
├── uf2conv.py               # Standalone binary to UF2 converter
├── uf2families.json         # Target family definitions (RP2350)
├── include/                 # Public API header interfaces
│   ├── gps.h               # PIO UART GPS receiver & NMEA parser
│   ├── lora.h              # REYAX RYLR998 UART1 driver
│   ├── navigation.h        # Waypoint guidance, dead-reckoning & telemetry
│   ├── payload.h           # GPIO16 solenoid release latch
│   └── propeller.h         # GPIO6 SG90 servo PWM mock propeller
├── src/                    # C source implementations (<= 8 lines/func)
│   ├── gps.c               # PIO0 SM0 initialization & NMEA RMC parsing
│   ├── lora.c              # UART1 configuration & ASCII transmission
│   ├── navigation.c        # Target coordinates, dead-reckon math & arrival
│   ├── payload.c           # Solenoid trigger & confirmation alerts
│   ├── propeller.c         # 50 Hz PWM driver & 150ms repeating timer
│   └── main.c              # Firmware entry point & 1-second dispatch loop
├── scripts/                # Host Python analysis & ground station tools
│   ├── decode_coordinates.py # Firmware binary coordinate scanner & patcher
│   ├── float_hex_converter.py # Interactive IEEE 754 float/hex decoder
│   └── telemetry_monitor.py  # Live ground station HUD for FT232RL bridge

```

4. High-Speed Propeller & PWM Architecture

The avionics firmware utilizes **Hardware PWM Slice 3 Channel A** on GP6 to drive an **SG90 Micro Servo** acting as a mock single-blade propeller.



Because a standard hobby SG90 servo possesses a physical mechanical limit (0° to 180°), the firmware simulates a spinning drone propeller by driving the internal motor back and forth between 0° ($1000\ \mu\text{s}$) and 180° ($2000\ \mu\text{s}$) at the maximum mechanical slew rate of the gear train. A hardware repeating timer (`add_repeating_timer_ms(-150, ...)`) flips the pulse width every **150 ms**, forcing the motor to operate at continuous maximum power without blocking the primary navigation loop.

When the craft reaches its target coordinates, `propeller_stop()` cancels the repeating timer and cuts the PWM output to zero, parking the propeller blade instantly upon payload release!

5. RP2350 Programmable I/O (PIO) Architecture & GPS Tutorial

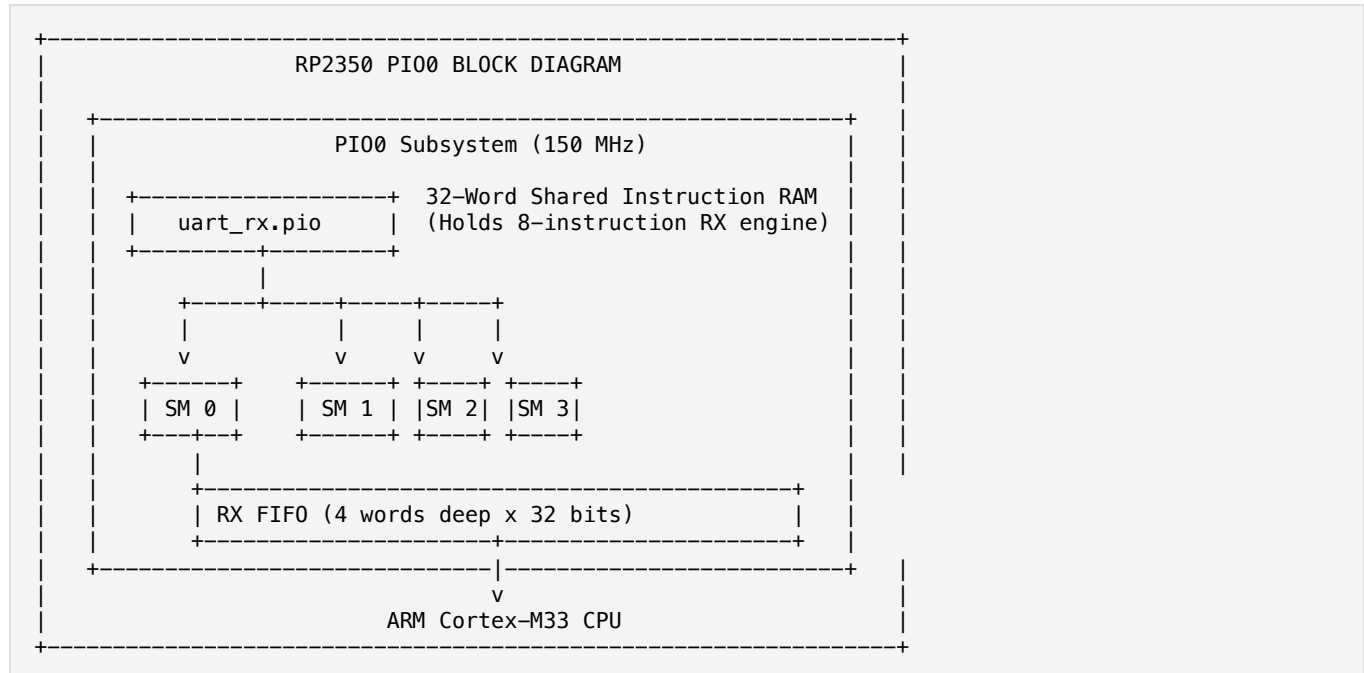
A critical engineering question arises when examining the flight computer: **How can the RP2350 communicate with three separate serial devices simultaneously when it only contains two hardware UART peripherals?**

The RP2350 micro-architecture incorporates two dedicated hardware UART blocks (`UART0` and `UART1`):

1. **UART0 (GP0 TX, GP1 RX):** Dedicated to the Raspberry Pi Debug Probe for serial interactive debugging and standard I/O (`printf`).
2. **UART1 (GP8 TX, GP9 RX):** Dedicated to the REYAX RYLR998 LoRa wireless transceiver running at 115,200 baud.

This exhausts all available hardware UART peripherals. To interface the **u-blox NEO-6M GPS receiver** at 9,600 baud on GP7, the avionics core leverages the RP2350's revolutionary peripheral subsystem:

Programmable I/O (PIO).



The `uart_rx.pio` Microcode

The PIO state machine executes a dedicated 8-instruction program (`uart_rx.pio`) clocked at 8 cycles per bit:

```
.program uart_rx
; 8 cycles per bit. IN pin 0 is RX line.
; Autopush disabled; pushes 8 bits manually.
    wait 0 pin 0      ; Wait for falling edge of START bit
    set x, 7          [10] ; 7 data bits to follow, delay to center of bit 0
bitloop:
    in pins, 1        ; Sample 1 bit from RX pin into Input Shift Register
    jmp x-- bitloop [6] ; Loop 8 times with 8-cycle timing between samples
    wait 1 pin 0      ; Wait for STOP bit (high line)
    push              ; Push 8-bit byte to RX FIFO
```

Clock divider calculation for 9,600 baud with 150 MHz system clock:

$$f_{\text{state_machine}} = 9600 \times 8 = 76,800 \text{ Hz}$$

$$\text{Clock Divider} = \frac{150,000,000}{76,800} \approx 1953.125$$

6. Curriculum Connection: Double Floating-Point Data Types

In **Week 5** (`0x0011_double-floating-point-data-type`), students analyzed the IEEE 754 64-bit double-precision floating-point standard:

$$V = (-1)^s \times 2^{e-1023} \times (1 + m)$$

Where:

- **Sign (s):** 1 bit (bit 63)
- **Biased Exponent (e):** 11 bits (bits 62 to 52, bias 1023)
- **Fractional Mantissa (m):** 52 bits (bits 51 to 0)

On 32-bit ARM Cortex-M33 architectures, 64-bit doubles cannot fit into a single general-purpose 32-bit core register (`r0` - `r12`). Instead, the ARM Architecture Procedure Call Standard (AAPCS) and Cortex-M33 instruction set manage doubles using **adjacent register pairs** (`r2`, `r3` or `r0`, `r1`) or 64-bit Floating-Point Unit (FPU) double registers (`d0` - `d15`).

When constants are declared in C:

```
const double TARGET_LAT = 38.881940;
const double TARGET_LON = -77.450280;
```

The compiler places the raw 8-byte IEEE 754 values into the literal pool in flash. At runtime, the Cortex-M33 executes a `ldrd` (Load Dual Register) instruction to fetch both 32-bit words into register pairs for floating-point comparison routines (`__aeabi_dcmpeq`).

7. Reverse Engineering in Ghidra (`0x0011a_cb`)

Step 1: Loading the Binary in Ghidra

Launch Ghidra, create a new project named `DarkVector`, and import `0x0011a_cb.bin` (or `0x0011a_cb.elf`):

- **Language / Processor:** `ARM:LE:32:Cortex` (ARMv8-M / Cortex-M33)
- **Base Address:** `0x10000000` (RP2350 External Flash Base)
- **Block Name:** `flash`
- **File Length:** 35,776 bytes

Step 2: Locating `main()` and Peripheral Inits

Tracing through the reset vector at `0x10000004` leads to `main()` at `0x1000084c`:

- `init_lora()`: Configures UART1 at 115,200 baud on `GP8 / GP9`.
- `init_gps_pio()`: Loads `uart_rx.pio` into `PI00 SMO` for 9,600 baud on `GP7`.
- `init_payload()`: Configures `GP16` as output LOW (Breadboard Red LED).
- `init_propeller()`: Sets up 50 Hz PWM on `GP6` (Divider 150.0, Wrap 19,999).

Step 3: Analyzing `navigate_to_target()` Disassembly

In Ghidra, function `navigate_to_target` resides at address `0x10000890`:

```

10000890: stmdb    sp!, {r4, r5, r6, r7, r8, r9, sl, fp, lr}
10000894: add     r7, pc, #148          ; Target Lat literal pool (0x1000092c)
10000896: ldrd    r6, r7, [r7]          ; Load TARGET_LAT (38.881940)
1000089a: addw    r9, pc, #152          ; Target Lon literal pool (0x10000934)
1000089e: ldrd    r8, r9, [r9]          ; Load TARGET_LON (-77.450280)
...
100008f4: bl      __aeabi_dcmpeq         ; Compare current_lat == TARGET_LAT
100008f8: cbz     r0, 10000916           ; In flight: branch to propeller_spin
10000902: bl      __aeabi_dcmpeq         ; Compare current_lon == TARGET_LON
10000906: cbz     r0, 10000916           ; In flight: branch to propeller_spin
10000908: bl      propeller_stop         ; Target reached: halt propeller!
10000912: b.w     release_payload        ; Target reached: release payload!
10000916: b.w     propeller_spin         ; Keep propeller oscillating

```

Step 4: Extracting the 64-bit Double Coordinates

Inspecting the literal pool words stored in flash at the end of `navigate_to_target`:

Address	32-bit Words (LE)	64-bit Hex Quadword
<code>0x1000092C</code>	<code>68F08462 404370E3</code>	<code>0x404370E368F08462</code> (Target Lat: +38.881940)
<code>0x10000934</code>	<code>633482BF C0535CD1</code>	<code>0xC0535CD1633482BF</code> (Target Lon: -77.450280)

(Note: The companion dead-reckoning module `dead_reckon_step` also accesses identical coordinates from literal pool addresses `0x10000858` and `0x10000860`.)

Step 5: Decoding Coordinates with `scripts/float_hex_converter.py`

Run the conversion tool directly on the quadwords found in Ghidra:

```
python3 scripts/float_hex_converter.py 0x404370E368F08462
```

Output:

```
[DECODE 0x404370E368F08462 (64-bit)]
1. Binary: 0 10000000100 0011011100001110001101101000111100001000010001100010
2. Sign: 0
3. Exp: 1028 - 1023 = 5
4. Value: 38.88194
5. ARM Regs: r3 (high) = 0x404370E3, r2 (low) = 0x68F08462
```

```
python3 scripts/float_hex_converter.py 0xC0535CD1633482BF
```

Output:

```
[DECODE 0xC0535CD1633482BF (64-bit)]
1. Binary: 1 10000000101 0011010111001101000101100011001101001000001010111111
2. Sign: 1
3. Exp: 1029 - 1023 = 6
4. Value: -77.45028
5. ARM Regs: r3 (high) = 0xC0535CD1, r2 (low) = 0x633482BF
```

Plotting (38.881940° N, 77.450280° W) identifies the intended target: **National Reconnaissance Office (NRO) Headquarters in Chantilly, Virginia!**

From the crash site in Centreville (38.840280° N, 77.428890° W), this target lies precisely **3.10 miles (4.99 km) NNW** at a flight vector bearing of **338.2°**.

8. The Binary Hack: Manual Ghidra Patching for Atlantic Disposal

To neutralize the threat, GMU engineers must overwrite the flight target directly in Ghidra with an offshore disposal zone in the **Atlantic Ocean** (37.000000° N, 74.000000° W, 226 miles offshore).

Step 1: Calculate Safe IEEE 754 Bytes with `scripts/float_hex_converter.py`

Run the converter on the safe latitude (37.0):

```
python3 scripts/float_hex_converter.py 37.0
```

Relevant 64-bit output:

- **Hex Quadword:** `0x4042800000000000`
- **Register Pair:** `r3 (high) = 0x40428000`, `r2 (low) = 0x00000000`
- **Little-Endian Bytes:** `00 00 00 00 00 80 42 40`

Run the converter on the safe longitude (-74.0):

```
python3 scripts/float_hex_converter.py -74.0
```

Relevant 64-bit output:

- **Hex Quadword:** `0xC052800000000000`
- **Register Pair:** `r3 (high) = 0xC0528000`, `r2 (low) = 0x00000000`
- **Little-Endian Bytes:** `00 00 00 00 00 80 52 C0`

Step 2: Manually Patch Memory Bytes in Ghidra

1. In the Ghidra **Listing view**, jump to address `0x1000092C` (file offset `0x092C`, `TARGET_LAT`).
2. Highlight the 8 bytes of the latitude quadword (`62 84 F0 68 E3 70 43 40`).
3. Press `Ctrl+Shift+G` (or right-click -> **Patch Data**).
4. Enter the replacement safe latitude bytes:

```
00 00 00 00 00 80 42 40
```

5. Jump to address `0x10000934` (file offset `0x0934`, `TARGET_LON` quadword: `BF 82 34 63 D1 5C 53 C0`).
6. Press `Ctrl+Shift+G` and enter the replacement safe longitude bytes:

```
00 00 00 00 00 80 52 c0
```

7. Repeat the patch for the dead-reckoning literal pool addresses at `0x10000858` (lat) and `0x10000860` (lon).

GHIDRA MEMORY PATCH VERIFICATION TABLE									
Address	Original (NR0 HQ Target)							Patched (Atlantic Ocean)	
0x1000092C	62	84	F0	68	E3	70	43	40	00 00 00 00 00 80 42 40
0x10000934	BF	82	34	63	D1	5C	53	C0	00 00 00 00 00 80 52 C0
0x10000858	62	84	F0	68	E3	70	43	40	00 00 00 00 00 80 42 40
0x10000860	BF	82	34	63	D1	5C	53	C0	00 00 00 00 00 80 52 C0

Step 3: Export Patched Binary from Ghidra

1. Navigate to **File -> Export Program...**
2. In the Export dialog:
 - **Format:** `Binary`
 - **Output File:** `0x0011a_cb_patched.bin`
3. Click **OK** to save the patched binary image to disk.

Alternatively, execute the automated forensic patch tool:

```
python3 scripts/decode_coordinates.py 0x0011a_cb.bin --patch
```

Step 4: Convert to UF2 and Verify Hardware Execution

Convert the exported binary to a bootable UF2 image using `uf2conv.py` :

```
python3 uf2conv.py 0x0011a_cb_patched.bin -f 0xe48bff59 -b 0x10000000 -c -o 0x0011a_cb_patched.uf2
```

Hold the **BOOTSEL** button on the Raspberry Pi Pico 2, copy `0x0011a_cb_patched.uf2` to the mass-storage volume, and monitor the UART console. The firmware output will confirm that the drone now tracks safely toward the Atlantic Ocean coordinates for offshore disposal:

```
CURRENT LAT: 38.840280, LON: -77.428890
TARGET LAT: 37.000000, LON: -74.000000
```

9. Ground Station Telemetry & Hardware Intercept (FT232RL & LoRa)

To monitor real-time drone telemetry without hardware serial bus contention, the RP2350 avionics assigns its communication interfaces according to the course hardware standard:

- **UART1 (GP8 TX, GP9 RX) @ 115200 baud:** Dedicated to the airborne **REYAX RYLR998 LoRa transceiver**. Upon power-up in the lab, the adversary's crash recovery beacon broadcasts long-range 915 MHz RF telemetry packets.
- **GPI06 :** Drives the SG90 servo mock propeller at 50 Hz PWM, oscillating continuously during flight and halting upon payload release.
- **GPI07 @ 9600 baud:** Dedicated to the **u-blox NEO-6M GPS receiver** via a single-wire PIO UART input (`PI00 SMO`).
- **UART0 (GP0 TX, GP1 RX) @ 115200 baud:** Dedicated to the **Raspberry Pi Debug Probe** serial console (`printf`).
- **GPI016 :** Controls the payload release latch, visually indicated in the lab by the **Red LED on the course breadboard**.

Intercepting the Adversary's Recovery Beacon (FT232RL to RYLR998)

Because the crashed drone begins transmitting its recovery beacon the moment GMU engineers power the board on the bench, investigators deployed a matching ground station node using a second REYAX RYLR998 module paired with an FT232RL USB-to-UART serial adapter:

[!IMPORTANT] Verify that the FT232RL jumper is configured for **3.3V** before connecting to the REYAX RYLR998 module. 5V logic will permanently damage the transceiver!

FT232RL USB Adapter	Ground RYLR998 LoRa	Electrical Description
VCC (3.3V Jumper)	VDD (Pin 1)	Regulated 3.3V DC Power
GND	GND (Pin 4)	System Common Ground
TXD (Data Out)	RXD (Pin 3)	Ground Station TX to LoRa RX
RXD (Data In)	TXD (Pin 2)	Telemetry Stream from LoRa TX

Real-Time Mission Telemetry Display (`scripts/telemetry_monitor.py`)

GMU engineers monitor the UAV's live navigation stream by running the Python ground station telemetry dashboard:

```
python3 scripts/telemetry_monitor.py --port /dev/tty.usbserial-0001 --baud 115200
```

Alternatively, run in simulated demonstration mode without hardware:

```
python3 scripts/telemetry_monitor.py --demo
```

The terminal displays a live 67-character ASCII telemetry HUD:

```
+-----+
|          DARK VECTOR TELEMETRY CONSOLE          |
| LINK: FT232RL / RYLR998 LORA GROUND STATION      | STATUS: ONLINE |
+-----+
| CURRENT POSITION : 38.840280 deg N  77.428890 deg W |
| TARGET WAYPOINT : 38.881940 deg N  77.450280 deg W |
| RANGE / BEARING  :    4.99 km / 338 deg            |
| PAYLOAD LATCH   : SECURED [GPIO16 LOW]           |
+-----+
```

Flashing the patched firmware (`0x0011a_cb_patched.uf2`) causes the HUD to reflect the new target waypoint, steering the craft safely out to sea:

```
+-----+
|          DARK VECTOR TELEMETRY CONSOLE          |
| LINK: FT232RL / RYLR998 LORA GROUND STATION      | STATUS: ONLINE |
+-----+
| CURRENT POSITION : 38.840280 deg N  77.428890 deg W |
| TARGET WAYPOINT : 37.000000 deg N  74.000000 deg W |
| RANGE / BEARING  :   363.74 km / 123 deg          |
| PAYLOAD LATCH   : SECURED [GPIO16 LOW]           |
+-----+
```

10. Student Deliverables Checklist

GMU engineering teams must complete the following exploitation steps:

#	Task Requirement	Verification Method	Deliverable
1	Locate Flash Vector Table	Read addresses <code>0x10000000</code> - <code>0x10000004</code>	Initial SP and Reset Pointer
2	Trace <code>main()</code> Function	Locate function at <code>0x1000084c</code>	Identify UART1 (GP8/GP9), PIO (GP7), GP16, GP6
3	Analyze PIO State Machine	Inspect <code>init_gps_pio()</code> call	Explain 8-cycle timing & 1953 clock divider
4	Extract Double Quadwords	Locate literal pool at <code>0x1000092C</code> / <code>0x10000934</code>	Raw hex bytes for NRO HQ Lat / Lon
5	Decode IEEE 754 Doubles	Run <code>scripts/float_hex_converter.py</code>	Target coordinates in decimal degrees
6	Identify Mock Propeller	Trace <code>init_propeller()</code> & PWM on GP6	Verify 50 Hz PWM & 150 ms repeating timer
7	Identify Solenoid Pin	Trace payload trigger <code>release_payload()</code>	Verify GP16 Red LED actuator pulse
8	Calculate Atlantic Hex	Run <code>float_hex_converter.py</code> on 37.0 / -74.0	Safe 8-byte hex replacement blocks

#	Task Requirement	Verification Method	Deliverable
9	Patch in Ghidra	Ctrl+Shift+G on literal pool offsets	Export 0x0011a_cb_patched.bin
10	Convert & Flash UF2	Run uf2conv.py	Prove Atlantic flight path via UART
11	Ground Station Telemetry	Run scripts/telemetry_monitor.py	Verify live coordinates and safe lock